

ESTRUCTURAS DE DATOS Y ALGORITMOS

RETO 1



OBSERVACIONES DE LA PRÁCTICA

Estudiante 1 Juan Manuel Gonzalez Cod 202613121

Estudiante 2 Larry Thomas Sanchez Cod 202615358

Estudiante 3 Jorge Claros Cod 202522281

REQUERIMIENTOS

1.

Estudiante: Juan Manuel González Claros.

Complejidad temporal: $O(N)$

Justificación: Cada iteración ejecuta únicamente operaciones de tiempo constante $O(1)$: comparar el producto, sumar a los acumuladores, comparar mínimos/máximos, y actualizar el conteo de años mediante el diccionario (cuyo acceso e inserción es $O(1)$ en promedio). Por lo tanto, la ejecución del ciclo principal realiza N iteraciones constantes por iteración.

2.

Estudiante: Larry Thomas Sanchez Beltran

N es el número total de pedidos cargados en el catálogo

Su complejidad es $O(N)$

En este requerimiento usamos array y es importante de mencionar que la complejidad en notación O se toma la parte con mayor complejidad en el código entonces para llegar a $O(N)$, nos concentramos en los recorridos en el que se aclara que no están anidados lo cual es la razón principal de que no sea $O(N^2)$, además el primer ciclo recorre los n pedidos del catálogo una sola vez y el segundo ciclo recorre una lista ya filtrada que tiene m elementos y en el peor de los casos $m = n$ y la complejidad sería $O(N)$ de todos modos, además de que los ciclos como ya se mencionó no están anidados o sea no está uno DENTRO de otro sino que va uno DESPUÉS del otro.

3.

Estudiante: Jorge Luis Claros Segura

Complejidad temporal: $O(N)$

Donde N es el número total de pedidos

Justificación:

Su ejecución se basa en un ciclo principal que itera N veces sobre el catálogo total de pedidos. Como la estructura base es un Array list, la búsqueda por índice (`get_element`) toma un tiempo constante $O(1)$. Los ciclos finales para hallar los máximos iteran sobre conjuntos limitados por N , resultando en una complejidad dominante lineal de $O(N)$

4.

Estudiante: Juan Manuel González

Complejidad temporal: $O(N + M^2)$

Justificación:

El código recorre primero el catálogo completo de N pedidos usando un `array_list`, donde el acceso por índice es $O(1)$, o sea es $O(N)$. Luego, para encontrar los dos pedidos de mayor Amount, recorre la lista de tamaño M usando un `single_linked_list`, pero accede por índice y como una lista enlazada no tiene acceso directo a una posición, cada llamada debe recorrer los nodos desde el inicio hasta esa posición, sumando $O(1+2+\dots+(M-1)) = O(M^2)$ y sumando las dos partes es: $O(N + M^2)$.

5.

Estudiante: Larry Thomas Sanchez Beltran

N es el número total de pedidos cargados en el catálogo

Su complejidad temporal es $O(N)$

Para este análisis podemos tomar en cuenta las mismas condiciones del requerimiento 2 porque se usó array y tiene dos ciclos que no están anidados y uno recorre los N pedidos según los filtros que se pongan en esta función que necesita de fechas y producto entonces ahí N varía según el filtro y el segundo ciclo recorre la lista filtrada, en ambos requerimientos si se analiza un supuesto peor, mejor o un promedio en casos aun así su complejidad sigue siendo $O(N)$ porque si o si hay que revisar los pedidos por lo menos una vez para verificar si cumplen o no el filtro entonces no cuenta con ningún atajo donde la complejidad cambie según el caso

6.

Estudiante: Jorge Luis Claros Segura

Complejidad temporal: $O(N*C)$

Donde N es el número total de pedidos en el catálogo y C es el número de canales distintos

Justificación:

El algoritmo recorre mediante un ciclo principal el catálogo de pedidos de tamaño N . si hay un total de C canales distintos acumulados, la búsqueda toma $O(C)$

Por lo tanto, la ejecución del ciclo principal realiza N iteraciones, donde cada una de ellas ejecuta operaciones de búsqueda de orden $O(C)$, resultando en una complejidad acumulada de $O(N*C)$

